Synapse Constraint Based Matchmaking for Hackathon Team Formation

Raghul M
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, Tamil Nadu, India
raghul.m.2024.cse@rajalakshmi.edu.in

Rithesh Madhav S
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, Tamil Nadu, India
rithesh.m.2024.cse@rajalakshmi.edu.in

Abstract—Hackathon platforms frequently expose participant profiles as free-text directories. Equivalent technical capabilities can therefore be represented by incompatible strings, which increases normalization effort and weakens the precision of team formation. This paper presents Synapse, a constraint-based matchmaking architecture that treats candidate selection as a Constraint Satisfaction Problem. The design uses finite database domains for project roles and experience levels, followed by a two-phase decision pipeline. First, role parity, experience floor, availability, and event compatibility are evaluated as deterministic hard constraints. Second, the feasible candidates are ranked using Jaccard similarity over structured technology sets and weighted contextual preferences. The platform is organized as a Next.js application with React Server Components, Prisma, PostgreSQL through Supabase and PgBouncer, NextAuth.js version five, and Vercel serverless deployment. The resulting model offers explainable rejection reasons, bounded categorical verification, and linear candidate filtering. The paper also identifies the limitations of rigid enumerations and outlines natural-language classification and GitHub-derived evidence as future extensions.

Keywords—constraint satisfaction, team formation, hackathons, finite domains, Jaccard similarity, Next.js, Prisma

I. INTRODUCTION

Hackathons impose a short and highly constrained team-formation interval. Participants must identify complementary roles, compatible experience levels, and a practical collaboration context before the event begins. Conventional portals make this task a directory search problem: a participant enters an arbitrary description, reviews many profiles, and infers whether a candidate can fill a team requirement. Such a process is inconsistent because React, React.js, react, and frontend development may denote related capability while remaining distinct database values.

The resulting ambiguity produces two costs. First, a system must repeatedly normalize or compare unbounded text representations. Second, users must perform manual pairwise screening, which can approach O(N^2) work when N profiles are repeatedly compared. Synapse reframes this activity as constrained allocation. It distinguishes feasibility from desirability: a candidate is admitted to the feasible set only when mandatory predicates hold, and is ranked only after that decision.

The principal contributions are a finite-domain data model, a deterministic hard-constraint filter, a set-based soft ranking function, and a server-side architecture that retains matching decisions behind authenticated application boundaries. The work draws on constraint-processing principles [1] and team-formation research [2].

II. SYSTEM ARCHITECTURE

A. Decoupled Application Topology

Synapse uses Next.js 14 as the application boundary. The App Router coordinates route segments and layouts, while React Server Components (RSCs) execute data-dependent rendering on the server. Client components are reserved for interactive filters, form controls, and stateful interface behavior. This separation keeps database credentials and matching predicates out of the browser while preserving a responsive search experience [3].

Route handlers and server actions receive requests, verify session state, validate payloads, and invoke Prisma. Prisma translates typed application operations to PostgreSQL queries. In the intended hosted configuration, Supabase provides PostgreSQL persistence and PgBouncer supplies connection pooling on port 6543. The pooled URL is appropriate for short-lived serverless invocations, whereas administrative tasks use a direct connection when required.

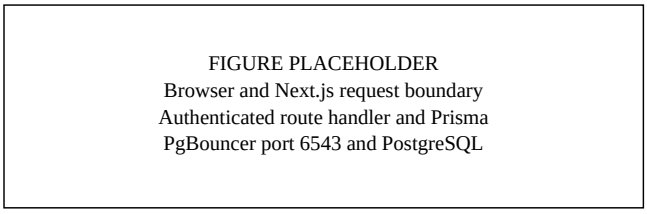


Fig. 1. Proposed Synapse deployment and data-access topology.

B. Serverless Execution Path

Vercel distributes static assets through its edge network and invokes dynamic code in isolated serverless executions. A request flows from a browser to a protected Next.js route, then to session verification, Prisma, the pooled database connection, and the database. This path centralizes authorization and prevents the client from issuing privileged persistence operations directly.

III. DATA MODELING AND FINITE DOMAINS

A finite domain makes a comparison meaningful before it is executed. The Prisma schema defines ProjectRole and ExperienceLevel as enumerations. The role domain includes FRONTEND, BACKEND, FULLSTACK, UI_UX_DESIGNER, DATA_SCIENTIST, AI_ML_ENGINEER, APP_DEVELOPER, and DEVOPS. The ordered experience domain contains BEGINNER, INTERMEDIATE, ADVANCED, and EXPERT. A user stores a primary role, experience level, technology-set array, and time-zone information; a hackathon requirement stores a required role, minimum experience, structured skill array, event mode, location, and open positions.

TABLE I
FINITE DOMAINS USED BY THE MATCHER

Attribute	Domain representation	Decision use
ProjectRole	Closed enum	Role parity
ExperienceLevel	Ordered enum	Minimum floor
Skills	Structured set	Jaccard ranking

Enumerations prevent a value such as “AI/ML engineer” from silently becoming a distinct category from AI_ML_ENGINEER. They do not eliminate all semantic work: technology labels remain a structured but extensible set because the long tail of frameworks evolves more quickly than the core role taxonomy. A uniqueness constraint on candidate and post identifiers additionally prevents duplicate applications.

IV. THE MATCHMAKING ALGORITHM

A. Constraint Satisfaction Formulation

Let U be the set of candidate profiles and let t denote a team requirement. The matching state is expressed as P equals X, D, C, where X contains candidate variables, D contains their finite domains, and C contains the mandatory predicates. The hard-filtered feasible set is formed by role equality, an ordinal experience comparison, environment compatibility, and availability compatibility. Each predicate is evaluated once per candidate. With indexed retrieval and constant-time categorical checks, the scan is O(N).

Role parity is evaluated as candidate role equals required role. Experience is mapped to the ordinal function phi, where phi(BEGINNER) is less than phi(INTERMEDIATE), phi(ADVANCED), and phi(EXPERT). A candidate is feasible only when phi(candidate experience) is greater than or equal to phi(required experience). This ordering turns a vague statement such as “senior enough” into a deterministic predicate and permits an explicit explanation when a candidate is excluded.

B. Soft Constraint Ranking

For each feasible candidate, Synapse compares the candidate technology set A with the requirement set B. Jaccard similarity measures the shared portion of the union and returns a value in the closed interval from zero to one [7].

$$J(A,B) = |A \cap B| / |A \cup B|, \quad (1)$$

The score is then combined with environment, availability, and optional-preference weights. An online event receives full remote-environment credit. An offline event requires compatible location, and a hybrid event may permit remote participation only when explicitly enabled. The weights alpha, beta, gamma, and delta are nonnegative and sum to one.

$$\text{Score}(u,t) = \alpha J(A,B) + \beta E(u,t) + \gamma V(u,t) + \delta P(u,t). \quad (2)$$

If K candidates remain after hard filtering and m is the bounded average technology-set size, the scoring stage is O(Km); ordering the result is O(K log K). The two-phase design prevents high technology overlap from compensating for a missing mandatory role or insufficient experience.

V. IMPLEMENTATION AND SECURITY

Authentication is implemented with NextAuth.js version five and OAuth 2.0 providers. The application uses JSON Web Token sessions, but a token is treated as evidence of authenticated state rather than as unrestricted authority. Middleware distinguishes public, authentication, and protected routes; unauthenticated requests to protected paths are redirected to login. Server-side handlers must still derive identity from the verified session and validate every mutation, because client-submitted identifiers are not authorization evidence [5].

Prisma is used as the typed persistence boundary and is paired with validation before data is committed. Environment variables hold provider credentials and database URLs; they are not exposed to client components. The deployment includes a custom .npmrc setting for legacy peer-dependency resolution, allowing Vercel builds to install the project dependency graph consistently. Locked versions, generated Prisma artifacts, and separate pooled versus direct database URLs are deployment controls rather than algorithmic assumptions.

VI. DISCUSSION AND LIMITATIONS

Finite domains permit O(1) categorical verification and make the filter explainable: a candidate can be told whether role, experience, location, or availability prevented a match. Database-side predicates also avoid loading a whole profile directory into the browser. The model is especially suitable for hackathons because mandatory team coverage is often more important than a globally optimized social score.

The same structure introduces rigidity. New specializations require a controlled schema migration, and a self-reported technology set is not independent evidence of competence. Jaccard similarity also treats skill labels uniformly unless domain-specific weights are introduced. Further, the current pairwise ranking does not solve the multi-member assignment problem, where several vacancies and candidates must be selected jointly. These limitations motivate human confirmation and later optimization stages rather than hidden automated rejection.

VII. CONCLUSION AND FUTURE WORK

Synapse defines a practical algorithmic basis for hackathon team formation. Finite roles and experience levels convert ambiguous input into comparable state; deterministic filtering identifies feasible candidates; and Jaccard-based ranking orders feasible alternatives without obscuring mandatory requirements. The architecture combines Next.js, RSCs, Prisma, PostgreSQL, NextAuth.js, and Vercel deployment so that data access and authorization remain server controlled.

Future work will use natural-language classification during onboarding, followed by user confirmation before any text is mapped to an enum or skill set. GitHub API integration may add opt-in public indicators such as language distribution and repository activity. The longer-term optimization problem is multi-member team assembly with role coverage, diversity, schedule overlap, and participant preference under explicit constraints.

ACKNOWLEDGMENT

The authors thank the Department of Computer Science and Engineering, Rajalakshmi Engineering College, for academic guidance and infrastructure support.

REFERENCES

[1] R. Dechter, Constraint Processing. San Francisco, CA, USA: Morgan Kaufmann, 2003.

[2] T. Lappas, K. Liu, and E. Terzi, “Finding a team of experts in social networks,” in Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2009, pp. 467–476.

[3] Vercel, “Next.js documentation,” 2026. [Online]. Available: https://nextjs.org/docs

[4] Prisma, “Prisma ORM documentation,” 2026. [Online]. Available: https://www.prisma.io/docs

[5] Auth.js, “Auth.js documentation,” 2026. [Online]. Available: https://authjs.dev

[6] Supabase, “Supabase database documentation,” 2026. [Online]. Available: https://supabase.com/docs

[7] P. Jaccard, “Etude comparative de la distribution florale dans une portion des Alpes et des Jura,” Bulletin de la Societe Vaudoise des Sciences Naturelles, vol. 37, pp. 547–579, 1901.